What Is SCAFFOLD?

One-Line Summary

SCAFFOLD (Stochastic Controlled Averaging for Federated Learning) is a federated learning algorithm that fixes a specific weakness of standard federated averaging (FedAvg): it corrects for 'client drift' when different clients hold different (non-IID) data, by adding a control-variate correction term to each client's local gradient during training.

Where It Comes From

Introduced by Karimireddy, Kale, Mohri, Reddi, Stich, and Suresh in "SCAFFOLD: Stochastic Controlled Averaging for Federated Learning" (ICML 2020). It is used in this project because the three simulated hospital clients have deliberately different pneumonia rates (12.5% / 24.4% / 17.7%) -- exactly the kind of heterogeneity SCAFFOLD is designed for.

Why It Matters Here

Without a correction like this, a client that sees mostly NORMAL cases (client_0, 12.5% pneumonia) and a client that sees far more PNEUMONIA cases (client_1, 24.4%) would each train toward their own, differently biased version of 'good enough,' and combining those biased models afterward would not reliably recover a model that is good for all three.

The Problem: Client Drift

How Plain FedAvg Works

In standard FedAvg, each round: the server sends the current global model to every client; each client trains on ONLY its own local data for several epochs; the server averages the resulting models (weighted by how much data each client has) to get the next global model.

Why That Breaks Under Non-IID Data

If every client had the same data distribution (IID), each client's local training would move the model in roughly the same direction, and averaging would work well. But when clients differ -- as in this project -- each client's local training pulls the model toward whatever pattern dominates ITS OWN data. Over several local epochs, this pull accumulates: each client ends up far from where it started, in a DIFFERENT direction from the other clients. This is called 'client drift.'

The Consequence

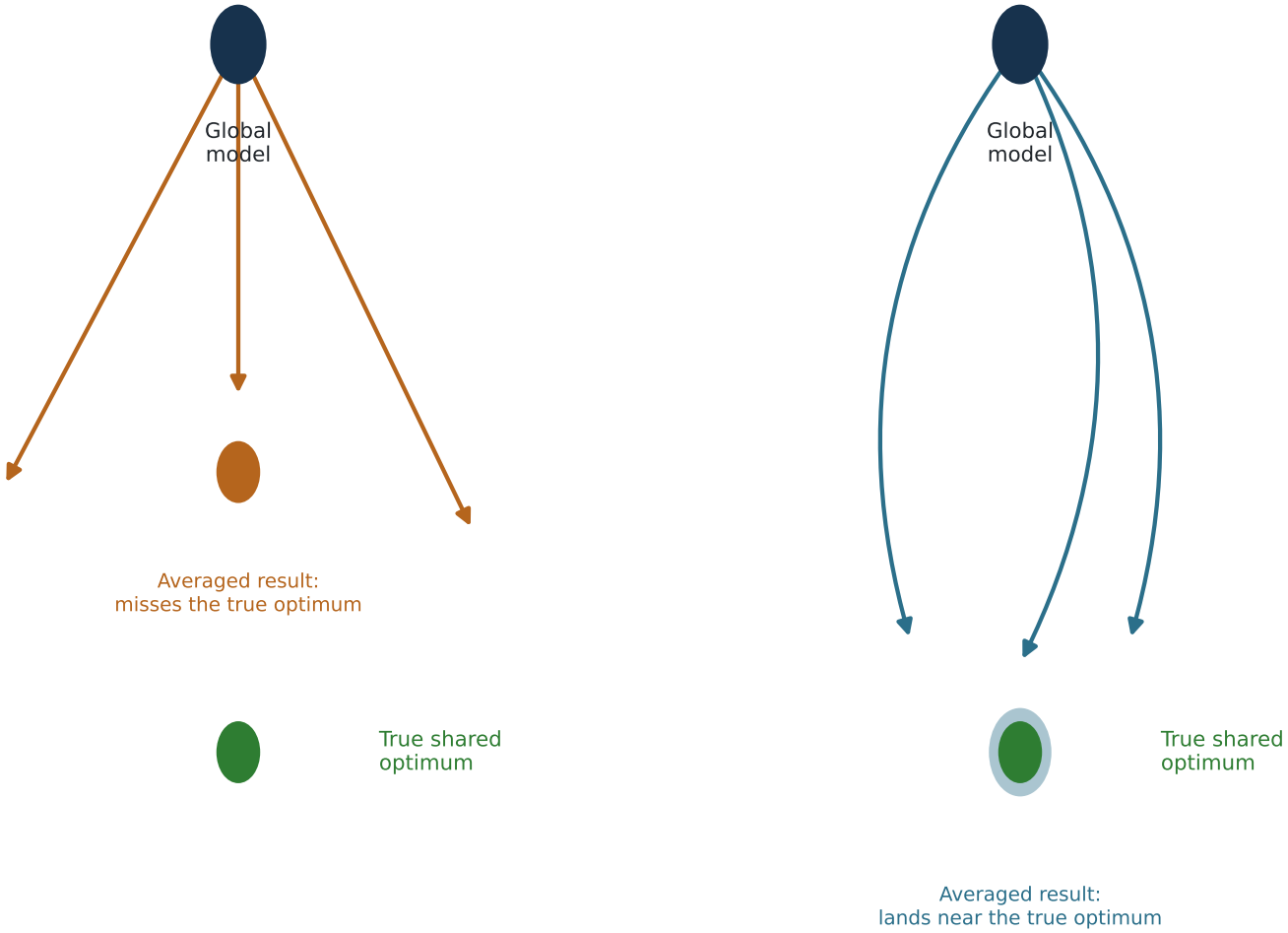
Averaging several models that have each drifted in different directions is a compromise, not a correction -- the result can be worse than any single client's model, and convergence slows down or becomes unstable as heterogeneity increases. The more communication rounds you run, the more this compounding drift can hurt, rather than help.

Visualizing Client Drift

Why non-IID client data pulls plain FedAvg off course, and how SCAFFOLD's correction pulls it back.

Without SCAFFOLD (plain FedAvg)

With SCAFFOLD (control-variate correction)



Each client's local training drifts toward ITS OWN data (amber, left). SCAFFOLD adds a correction term to every local gradient step that continuously steers each client's path back toward the federation-wide direction (teal, right) -- not just a one-time fix at the end.

The Core Idea: Control Variates

Intuition

Think of each client as a hiker trying to walk toward a destination, but a local 'wind' (its own data's bias) keeps blowing it off course. SCAFFOLD gives each hiker a compass correction: an estimate of (a) which way THIS hiker's local wind tends to blow, and (b) which way the whole group, on average, needs to walk. Subtracting (a) and adding (b) at every step keeps each hiker walking in the shared direction instead of drifting with their own local wind.

The Two Control Variates

c_local (written c_i for client i): an estimate of the direction THIS client's own data pulls training toward. Each client keeps and updates its own c_i, round after round.

c_global (written c): an estimate of the direction the WHOLE FEDERATION, on average, pulls training toward. Maintained by the server and shared with every client.

The Correction Formula

At every local training step, instead of using the plain gradient g, each client uses a corrected gradient:

```
g_corrected = g_local + (c_global - c_local)
```

The term (c_global - c_local) is exactly the client-specific bias that plain FedAvg leaves uncorrected: it is added to every single local update, for the whole local training run, not just once at the end of the round.

The Algorithm, Step by Step

Step 1 - Broadcast

The server sends the current global model AND the current global control variate (c) to every participating client.

Step 2 - Local Corrected Training

Each client trains locally for a fixed number of epochs. At every gradient step, it computes the normal loss gradient, then adds the correction (c - c_i) before updating its weights.

Step 3 - Update the Local Control Variate

After local training finishes, each client updates its own c_i so that it better reflects the direction its data actually pulled training toward this round. This project uses the paper's 'Option II' update, which infers this from how far the model's parameters actually moved:

c_i_new = c_i_old - c + (x_before - x_after) / (local_steps x learning_rate)

where x_before is the model at the start of the round and x_after is the model after local training. (The paper's 'Option I' alternative computes this more directly, via an extra gradient pass on the original model, at extra computational cost -- Option II is the cheaper, commonly used choice, and the one implemented here.)

Step 4 - Send Updates Back

Each client sends its updated model back to the server (as in standard FedAvg), along with information the server needs to update the global control variate.

Step 5 - Server Aggregation

The server averages the returned models, weighted by each client's number of training samples (same as FedAvg), AND updates the global control variate c by combining the changes reported by each client. The updated global model and updated c are broadcast for the next round.

Why This Actually Helps

Continuous, Not One-Time, Correction

The key mechanical detail is that the correction (c - c_i) is applied at EVERY local gradient step, for every epoch of local training -- not as a single adjustment at the end of the round. This keeps each client's entire local training trajectory anchored to the federation-wide direction, instead of letting it drift for a while and trying to fix it afterward.

What the Original Paper Shows

Karimireddy et al. prove that SCAFFOLD converges reliably under client heterogeneity where plain FedAvg's convergence guarantees weaken or break down, and that SCAFFOLD's convergence rate does not get worse as data becomes more non-IID -- a property FedAvg does not have.

In Plain Terms

SCAFFOLD does not change WHAT each client learns from its own data. It changes HOW that local learning is steered, so that combining three differently-biased clients still reliably points toward one shared, useful model -- which is exactly the situation this project's three non-IID clients create.

How This Project Implements SCAFFOLD

Where It Lives in the Code

train_scaffold_updated.py's ScaffoldClient class: each simulated client keeps its own c_local and c_global as lists of tensors (one per model parameter), applies the correction inside its local training loop, and updates c_local using the Option II formula after each round.

Framework

Implemented on top of Flower (flwr), which handles the client/server communication and simulation of 3 virtual clients, with PyTorch used for the actual model training.

An Honest Scope Note

The current implementation keeps c_global fixed at zero rather than aggregating a true, round-by-round updating global control variate from all clients. In practice, this means each client's correction term reduces to (-c_local): it still cancels that client's OWN accumulated drift round over round, which is a meaningful and effective simplification, but it does not additionally pull each client toward the other clients' directions the way a fully-aggregated global c would. This is a deliberate scope simplification of the current baseline, not an oversight -- worth knowing if asked about it directly, since the full algorithm's cross-client correction is the part that is simplified here.

Result

Even with this simplification, the SCAFFOLD-based model reaches 80.79% accuracy, 80.77% precision, 80.83% recall, and 80.80% F1 on the held-out test set, consistent across three independent training seeds.

Summary: Key Points to Remember

1. Problem: non-IID client data causes 'client drift' in plain FedAvg -- each client's local training biases toward its own data, and averaging drifted models afterward is a compromise.

2. Idea: SCAFFOLD adds a control-variate correction, (c_global - c_local), to every local gradient step, continuously steering each client back toward the shared, federation-wide direction instead of its own local bias.

3. c_local estimates 'which way this client's own data pulls'; c_global estimates 'which way the whole federation, on average, pulls.'

4. Each round: broadcast model + c -> local corrected training -> update c_local -> return model + control-variate info -> server aggregates both the model and c.

5. This project uses SCAFFOLD specifically because its 3 simulated clients have different pneumonia rates (12.5% / 24.4% / 17.7%) -- a real non-IID scenario, not a hypothetical one.

6. Reference: Karimireddy et al., "SCAFFOLD: Stochastic Controlled Averaging for Federated Learning," ICML 2020.